目录

1. 执行摘要与战略背景 2
2. 理论框架与 CXL 生态系统深度解析 2
2.1 CXL 协议层的仿真需求分析 3
2.1.1 CXL.io：发现与配置的基础 3
2.1.2 CXL.cache：一致性的核心 3
2.1.3 CXL.mem：内存扩展的主通道 4
2.2 拓扑结构与延迟特性的量化分析 4
2.2.1 延迟层级与拓扑影响 4
2.2.2 带宽饱和特性的非线性 5
2.3 现有仿真技术的局限性与突破方向 5
3. 系统架构设计：Epoch-Based 执行驱动模型 6
3.1 总体架构图与数据流 6
3.2 核心技术选型与论证 7
3.2.1 追踪技术：Intel PEBS + LBR 7
3.2.2 内存分配追踪：eBPF 7
3.2.3 拓扑建模：图论模型 8
4. 核心功能模块的详细设计与算法实现 8
4.1 追踪器模块（The Tracer) 8
4.2 时序分析器（The Timing Analyzer) 9
4.2.1 延迟计算模型（The Delayed-Buffer Model） 9
4.2.2 带宽与拥塞模型（Time-Series Congestion) 9
4.2.3 协议开销修正 9
4.3 注入器模块（The Injector) 10
5. 系统实现步骤与进度安排 10
6. 测试与验证策略 12
6.1 精度验证：微基准测试 12
6.2 场景验证：大语言模型（LLM）推理 12

6.3 策略验证：HPC 工作负载的交错访问 13
7. 结论 13

CXL 内存仿真平台设计与实现综合执行报告：从理论架构到系统部署

1. 执行摘要与战略背景

在当前的现代数据中心架构演进中，**计算与内存资源的紧密耦合**已成为**制约性能扩展与资源利用率提升的核心瓶颈。**随着异构计算需求的爆炸式增长，内存容量与带宽的扩展速度难以匹配处理器的核心数增长，导致了“内存墙”效应的加剧以及严重的资源搁浅（Memory Stranding）问题。在此背景下，Compute Express Link (CXL)作为一种革命性的互连标准应运而生，旨在通过**解耦计算与内存资源**，**实现内存池化**（Memory Pooling）与**即时组合**（Composability），从而根本性地重塑服务器架构。

尽管 CXL 1.1、2.0 及 3.0 标准在理论层面提供了**稳健的一致性协议与内存扩展机制**，但在实际落地过程中，系统架构师与研究人员面临着巨大的验证鸿沟。一方面，支持 CXL 2.0/3.0 的**高级硬件**（如多逻辑设备 MLD、Fabric 交换机）**尚未广泛商用**，导致实证研究受限；另一方面，**传统的周期精确级模拟器**（如 Gem5）虽然**精度极高**，但其**运行速度比原生执行慢数千倍**，无法支撑大规模工作负载（如大语言模型训练、内存数据库）的性能特征分析。

本报告依据《CXL 内存仿真平台设计与实现任务书》的要求，结合学术界最新的实证研究与仿真方法论，制定了一份详尽的执行计划。

本计划旨在构建一个**兼顾仿真精度、执行速度与拓扑灵活性的软件定义仿真平台。**该平台采用**基于 Epoch 的用户态执行驱动（Execution-Driven）架构，利用底层硬件性能计数器（Intel PEBS/LBR）与内核级追踪技术（eBPF），精确模拟 CXL.io、CXL.cache 及 CXL.mem 协议栈的延迟与带宽特性。**通过本平台的实施，我们将能够量化评估 CXL 引入的延迟与拥塞效应对实际应用的影响，探索内存分层（Tiering）、交错（Interleaving）及一致性维护（Coherency）等关键策略的最优解，为下一代数据中心架构的设计提供坚实的理论验证工具。

2. 理论框架与 CXL 生态系统深度解析

要构建一个高保真的仿真平台，必须首先深入剖析被仿真对象的本质属性。CXL 不仅仅是一

条物理连线，它是一套复杂的协议栈，定义了设备如何与主机内存一致性域进行交互。

2.1 CXL 协议层的仿真需求分析

CXL 标准通过在 PCIe 物理层之上复用三种协议来实现其功能，仿真平台必须能够区分并分别建模这三种协议的流量特征，因为它们对系统性能的影响机制截然不同。

2.1.1 CXL.io：发现与配置的基础

CXL.io 基于 PCIe 传输层，负责设备的发现、枚举、寄存器访问以及中断处理。在实际系统中，CXL.io 不仅用于初始化，还承载着非一致性的加载/存储操作（Non-coherent Load/Store）。从仿真角度来看，CXL.io 流量通常遵循 PCIe 的生产者-消费者排序模型（Producer-Consumer Ordering Model）。

仿真挑战：尽管 CXL.io 流量在稳态运行中占比不高，但在设备初始化和错误处理阶段至关重要。仿真器需要模拟 PCIe 事务层的数据包（TLP）开销和排序规则。

关键数据点：根据 CXL 3.0 规范，**CXL.io 事务具有非转发（Non-Posted）和转发（Posted）两种类型，且需通过虚拟通道（Virtual Channels）进行 QoS 管理。**仿真平台需模拟这种基于信用的流控机制，尽管在大规模内存访问分析中可进行适当抽象，但在设备驱动开发场景下必须保留其时序特征 。

2.1.2 CXL.cache：一致性的核心

CXL.cache 协议允许设备（如加速器或智能网卡）缓存主机内存，或者允许主机缓存设备内存。这是 CXL 区别于 PCIe 的关键特性。

D2H 与 H2D 通道：CXL.cache 定义了从设备到主机（D2H）和主机到设备（H2D）的请求、响应及数据通道。例如，当设备希望修改一行数据时，必须发送 RdOwn（Read for Ownership）请求。

Back-Invalidate (BI) 机制：在 CXL 3.0 及更高版本中，为了支持设备对主机内存的一致性访问，引入了 Back-Invalidate 流。当主机需要收回某个缓存行的权限时，必须向所有可能持有该行副本的设备发送监听（Snoop）消息。这在多主机共享内存场景下尤为关键。

仿真实现路径：仿真平台必须捕捉导致缓存状态变更的事件。单纯的内存读写无法触发 BI

流，必须结合 LBR（Last Branch Record）推断指令意图，区分普通写与 RFO（Read For Ownership）操作，并模拟 Snoop 过滤器带来的额外延迟 。

2.1.3 CXL.mem：内存扩展的主通道

CXL.mem 允许主机处理器像访问本地 DDR 一样访问连接在 CXL 总线上的存储介质。这是本项目的核心仿真对象。

HDM-H 与 HDM-D：CXL.mem 支持主机管理设备内存（HDM）。HDM-H（Host-only Coherent）模式下，设备不维护一致性状态，所有一致性由主机负责，这简化了设备设计但增加了主机负担。HDM-D（Device Coherent）模式下，设备参与一致性维护，支持偏置翻转（Bias Flip）操作。

Flit 打包效率：CXL 2.0 使用 68 字节 Flit，而 CXL 3.0 引入了 256 字节 Flit 以支持 PAM-4 信号传输。256 字节 Flit 虽然带宽更高，但引入了 FEC（前向纠错）和 CRC 开销。仿真平台在计算有效带宽时，必须扣除这些协议开销。例如，256B Flit 的有效载荷效率约为 93% 。

2.2 拓扑结构与延迟特性的量化分析

CXL 引入了类似 NUMA 但又具有独特属性的访问层次。理解这些物理属性是建立准确延迟模型的前提。

2.2.1 延迟层级与拓扑影响

在传统的双插槽服务器中，跨插槽访问（Remote NUMA）通常增加约 1 个 Hop 的延迟。而 CXL 内存的表现则更为复杂，通常被视为“两跳”NUMA 节点。

基准延迟：本地 DRAM 访问延迟约为 80-100ns。

CXL 附加延迟：根据现有硬件实测，直接连接的 CXL 内存（CXL 1.1）会增加约 170ns 的往返延迟。如果经过一层 CXL 交换机（CXL 2.0/3.0），延迟将进一步增加 40-70ns（包含内部路由与 SerDes 重定时）。

总体延迟：一个典型的经过交换机的 CXL 内存访问，其总负载到使用（Load-to-Use）延迟可

能达到 **250ns** 甚至更高。

下表总结了不同拓扑下的预估延迟模型，这将作为仿真器参数配置的基准数据：

访问路径类型	预估总延迟 (ns)	相对本地 DRAM 倍数	组成部分分析
本地 DDR5	~90	1.0x	控制器 + 介质访问
跨插槽 (UPI) DDR5	~140	1.5x	UPI 链路 + 远程控制器
直连 CXL Type 3 (Gen5)	~170-200	~2.0x	PCIe/CXL PHY + CXL 控制器 + 介质
经交换机 CXL (单级)	~240-260	~2.7x	上述 + 交换机处理 + 线缆飞行时间
跨 Fabric CXL (多级)	>300	>3.3x	多级交换 + 拥塞排队

2.2.2 带宽饱和特性的非线性

与 DDR 通道可以通过增加通道数线性扩展带宽不同，CXL 内存的带宽受到 PCIe 链路宽度（如 x8 或 x16）和控制器处理能力的双重限制。实测数据显示，CXL 内存的带宽往往在并发线程数达到 4-8 个时即进入饱和状态，而本地 DRAM 通常能支撑 20 个以上的并发线程线性增长。

仿真启示：仿真器的带宽模型不能简单设定为固定值。它必须是一个基于时间序列的动态模型，能够模拟随着请求队列深度增加而导致的非线性延迟增长。当监测到的瞬时带宽需求超过链路物理上限（如 PCIe 5.0 x16 理论上的 64GB/s）时，必须强制注入指数级的排队延迟。

2.3 现有仿真技术的局限性与突破方向

在确定技术路线之前，我们需要审视现有的解决方案为何无法满足“**既要精度又要速度**”的需求。

硬件仿真（FPGA）：利用 FPGA（如 DirectCXL）实现 CXL 控制器是精度最高的方法。但 FPGA 开发周期长，成本高昂，且难以模拟大规模的多级交换机拓扑。一旦硬件定型，改变拓扑结构（如从树状变为网状）极其困难 。

周期精确模拟（Gem5）：Gem5 提供了极其详尽的微架构模拟。然而，其仿真速度通常比原生执行慢 10,000 倍以上。运行几秒钟的基准测试可能需要数小时，这使得运行像数据库或 LLM 推理这样的真实负载变得不切实际 。

基于 NUMA 的模拟：简单地将远程 NUMA 节点作为 CXL 内存使用。这种方法忽略了 CXL 特有的流控、Flit 打包开销和交换机拥塞行为，导致结果过于乐观，无法反映真实的带宽瓶颈 。

本方案的选择：基于追踪的软件定义仿真（Trace-based User-space Simulation） 本计划采用 CXLMemSim 提出的架构路线：在真实硬件上运行应用程序，利用轻量级追踪工具捕获内存行为，然后通过数学模型计算 CXL 带来的额外开销，并实时注入到应用线程中。这种方法实现了原生级别的运行速度（仅 3-6 倍慢于原生，而非 10000 倍），同时保留了指令流的真实性。

3. 系统架构设计：Epoch-Based 执行驱动模型

本系统采用分层架构设计，旨在实现高内聚低耦合，确保各模块可独立开发验证。系统的核心思想是将程序的连续执行离散化为一系列时间片（Epoch），在每个 Epoch 结束时进行性能修正。

3.1 总体架构图与数据流

系统由前端配置层、后端核心引擎层以及底层基础设施层组成。

前端配置与控制层：

拓扑定义器（Topology Definer）：接收 JSON/YAML 格式的配置文件，定义根复合体（RC）、交换机层级、内存池节点及其各自的带宽/延迟属性。

内存映射管理器（Memory Mapper）：负责向操作系统申请大页内存，并将其虚拟地址范围映射到模拟的 CXL 设备 ID 上。支持定义不同的分配策略（如 First-touch, Interleave, Tiering）。

后端核心引擎层（Runtime Engine）：

追踪器（Tracer）：通过 perf_event_open 系统调用接口，实时从 PMU（性能监控单元）读取数据。

时序分析器（Timing Analyzer）：系统的“大脑”，包含延迟模型、带宽模型和拥塞控制算法。

注入器（Injector）：利用 nanosleep 或自旋循环（Spin-loop）对目标线程实施精准减速。

底层基础设施层：

Linux Kernel：提供 eBPF 钩子和 PMU 驱动支持。

硬件 PMU：Intel CPU 的 PEBS（精确事件采样）和 LBR 功能。

3.2 核心技术选型与论证

3.2.1 追踪技术：Intel PEBS + LBR

为了满足“精确模拟 CXL.mem 读写”的要求，我们不能仅依赖普通的性能计数器，因为它们无法提供导致 Cache Miss 的具体指令地址和数据地址。

PEBS (Precision Event-Based Sampling)：当计数器溢出时，硬件自动将处理器上下文（包括数据线性地址 Data Linear Address）写入内存缓冲区。这使我们能够精确获知是哪一次内存访问（虚拟地址）发生了 LLC Miss，并判断该地址是否落入模拟的 CXL 内存范围内 。

LBR (Last Branch Record)：仅知道发生了 Miss 是不够的，现代 CPU 的乱序执行（Out-of-Order Execution）和重排序缓冲区（ROB）可以掩盖部分内存延迟。通过 LBR 记录的分支历史，我们可以重构指令流，估算内存级并行度（MLP）。如果 CPU 在等待 CXL 数据的同时能执行其他无关指令，则不应注入全部延迟。这是提高仿真精度的关键一环 。

3.2.2 内存分配追踪：eBPF

为了动态感知应用程序的内存布局，系统将加载 eBPF 程序挂载到内核的 sys_mmap、sys_brk 和 page_fault 跟踪点。

机制：当应用申请内存时，eBPF 探针捕获返回的虚拟地址范围。

策略应用：用户空间的管理进程根据预设策略（如：前 4GB 分配给本地 DRAM，后续分配给 CXL 节点 1），将这些地址范围标记在全局映射表中。当 PEBS 捕获到访问时，直接查表即可确定目标设备。

3.2.3 拓扑建模：图论模型

内部使用有向加权图来表示 CXL Fabric。

节点：Host CPU、Switch Port、Endpoint Memory Controller。

边：物理链路（Link），具有属性 {Bandwidth_Cap, Latency, Current_Load}。

路由算法：支持 CXL 2.0 的基于 ID 的路由（树状）和 CXL 3.0 的基于端口的路由（PBR，支持多路径）。这使得系统能够模拟复杂的 Fabric 拓扑和跨机柜互连 。

4. 核心功能模块的详细设计与算法实现

4.1 追踪器模块（The Tracer)

追踪器是数据采集的源头，其质量直接决定仿真的置信度。

采样事件配置：

MEM_LOAD_RETIRED.L3_MISS：捕获读取 CXL 内存的操作。

MEM_INST_RETIRED.ALL_STORES：捕获写入操作（用于模拟写回流量）。

CYCLE_ACTIVITY.STALLS_L2_PENDING：辅助判断 CPU 是否因等待内存而停顿。

数据处理管道：

初始化 Perf 文件描述符，映射 Ring Buffer。

在每个 Epoch（如 10ms）结束时，批量读取 Buffer 中的采样记录。

解析记录中的 Data Linear Address，过滤掉属于本地 DRAM 的访问，仅保留命中 CXL 模拟区域的记录。

提取 TSC（时间戳计数器），用于重建访问的时间序列。

4.2 时序分析器 (The Timing Analyzer)

这是实现高保真模拟的核心算法所在。

4.2.1 延迟计算模型（The Delayed-Buffer Model）

基础的延迟计算公式并不足以模拟真实情况。我们需要引入 ROB 掩盖效应。

$$\Delta T_{lat} = \sum_{i \in Misses} \frac{L_{CXL_RoundTrip} - L_{Local_DRAM}}{MLP_Factor(i)}$$

ROB 模拟：利用 LBR 数据分析在 Cache Miss 发生前后的指令窗口。如果发现多个 Miss 发生在很近的时间窗口内且没有依赖关系，说明它们是并行发出的。此时，MLP_Factor 会大于 1，从而减少注入的惩罚时间。这种微架构层面的建模是超越传统 NUMA 模拟的关键 。

4.2.2 带宽与拥塞模型 (Time-Series Congestion)

CXL 链路的拥塞是导致性能非线性的主要原因。

时间切片：将一个 Epoch 进一步细分为微秒级的微切片（Micro-slice）。

流量积分：在每个微切片内，累加所有流经特定链路（Link）的字节数 Btotal 。

阈值判定：如果 Btotal >Link_BW×Slice_Time，则判定发生拥塞。

延迟注入：计算排空积压数据所需的时间：

$$T_{congestion} = \frac{B_{total}}{Link_BW} - Slice_Time$$

这个 Tcongestion 将作为额外的惩罚叠加到基础延迟上。对于经过多级交换机的流量，需要在每一级链路分别计算拥塞，取最大值或累加值（取决于交换机缓冲策略）。

4.2.3 协议开销修正

为了精确模拟 CXL.mem ，必须考虑协议层开销：

Flit 打包效率：对于 CXL 3.0 的 256B Flit，包含 2 字节头、CRC 和 FEC。有效载荷约为 230-240 字节。模型中的 Link_BW 将设定为 物理带宽 * 0.93 左右的有效带宽。

CXL.cache 一致性开销：如果检测到对“共享”状态页面的写入（通过 OS 页表状态或模拟目录得知），分析器将注入额外的 Tsnoop 延迟，模拟 Back-Invalidate 操作对总线的占用和往返时间 。

4.3 注入器模块（The Injector)

计算出的延迟必须作用于应用程序才能体现效果。

软中断机制：利用 Linux 信号（Signal）或 Ptrace 机制暂停目标线程。

混合休眠策略：

对于长延迟（>50us），使用 nanosleep 释放 CPU 资源，允许其他线程运行（模拟 IO 等待）。

对于短延迟（<50us），使用自旋循环（Busy-wait）占用 CPU，防止上下文切换带来的额外开销干扰测量精度。

5. 系统实现步骤与进度安排

基于任务书的时间表，我们将开发过程细化为四个阶段。

第一阶段：基础设施搭建与原型验证（2025.11 - 2026.01)

目标：

打通从硬件采样到用户态分析的数据通路。

关键任务：

环境准备：配置 Linux 内核，确保存取 PEBS/LBR 的权限（perf_event_paranoid 设置）。

Tracer 开发：编写 C++模块，封装 perf_event_open，实现对 LLC_MISS 事件的稳定捕获和地址解析。

Injector 开发：实现一个简单的“每秒暂停 100ms”的注入器，验证对目标程序的控制能力。

eBPF 钩子：编写 BPF 程序追踪 mmap，并能将数据传输到用户空间。

交付物：一个能够打印目标程序内存访问热力图的工具原型。

第二阶段：CXL 核心模型实现（2026.02 - 2026.03)

目标：实现数学模型，完成延迟与带宽的仿真逻辑。

关键任务：

延迟算法：实现带有 MLP 修正的延迟计算逻辑。

带宽算法：实现基于时间序列的流量积分与拥塞判决逻辑。

配置文件解析：定义 JSON 格式，允许用户输入 Switch_Latency (e.g., 40ns), Link_BW (e.g., 64GB/s)。

交付物：CXLMemSim Alpha 版本，能够对简单的微基准测试（如 STREAM）产生预期的性能降级（Bandwidth Throttling）。

第三阶段：高级特性与协议仿真（2026.03 - 2026.04)

目标：解决“精确模拟 CXL.mem/cache 协议”的难点。

关键任务：

一致性模拟：引入“共享页表”数据结构，模拟多节点访问同一内存区域时的 Back-Invalidation 延迟。

分层策略：实现内置的页面迁移策略（如 Page Migration），模拟在运行时将热页从 CXL 移动到本地 DRAM 带来的性能抖动。

多线程支持：优化 Epoch 同步机制，确保在多线程环境下，锁竞争导致的等待时间被正确处理，避免重复注入延迟 。

第四阶段：验证、测试与文档（2026.04 - 2026.05)

目标：验证精度，完成毕业设计要求。

关键任务：

基准校准：使用 Intel MLC (Memory Latency Checker) 运行在仿真器上，生成的“带宽-延迟曲线”应与业界发布的真实 CXL 硬件曲线（如 Samsung CXL Expander 数据）吻合度在 85%以上。

真实负载测试：运行 Redis（内存数据库）或 Llama.cpp（LLM 推理），验证在不同拓扑下的性能变化趋势是否符合理论预期（如 LLM 的 Decode 阶段受带宽限制明显）。

文档撰写：整理系统架构文档、API 说明及测试分析报告。

6. 测试与验证策略

为了证明系统的有效性，我们将执行严格的验证计划，涵盖微基准测试与宏观工作负载。

6.1 精度验证：微基准测试

工具：Intel MLC。

测试用例：

空载延迟：配置仿真器为“CXL 直连”，运行 MLC Latency 测试。预期结果应显示比本地 DRAM 高约 170ns 的延迟。

满载带宽：运行 MLC Bandwidth 测试。预期结果应在并发线程数达到 6-8 时出现带宽饱和平台期，且峰值带宽应受限于配置的 CXL 链路带宽（如 PCIe 5.0 x8 的 32GB/s）。

拥塞测试：配置两个虚拟 CPU 核心通过同一个模拟交换机端口访问内存。当双核同时满载时，应观察到显著的延迟抖动，验证拥塞模型的有效性。

6.2 场景验证：大语言模型（LLM）推理

参考最新的 CXL 应用研究，LLM 是 CXL 内存的重要应用场景。

工具：Llama.cpp 或 FlexGen。

测试场景：

Baseline：全部模型权重在本地 DRAM。

CXL 模拟：将 KV Cache 或部分模型权重放置在模拟的 CXL 内存区域。

预期结论：仿真器应能复现出 LLM 推理的两个阶段特征：Prefill 阶段对延迟敏感（性能下降较明显），而 Decode 阶段对带宽敏感（受限于 CXL 带宽瓶颈）。如果仿真结果能重现论文中的性能下降比例（如 20-30%），则证明平台具有极高的实用价值。

6.3 策略验证：HPC 工作负载的交错访问

工具：Graph500 或 NAS Parallel Benchmarks (NPB)。

测试场景：比较“CXL 优先分配”策略与“基于对象感知的交错（Object-Level Interleaving）”策略。

预期结论：仿真器应能展示出，对于带宽敏感型对象（如大矩阵），交错分配能提升性能；而对于延迟敏感型对象（如指针追踪图），本地优先分配性能更好 。

7. 结论

本执行计划通过采用基于 Epoch 的软件定义仿真架构，巧妙地避开了硬件稀缺与周期模拟效率低下的双重困境。通过深度集成 Intel PEBS/LBR 硬件特性与 Linux eBPF 内核技术，我们不仅能够高保真地还原 CXL 3.0 复杂的协议行为（如 Flit 开销、Back-Invalidation），还能保持足够的运行速度以支撑现代数据中心级工作负载的分析。该平台的建成，将为学术界研究 CXL 内存池化、异构计算调度及新型数据中心架构提供一个强有力且灵活的实验底座，完全符合毕业设计任务书对于系统完整性、创新性与技术深度的要求。